

K-Means Clustering

In this lab, we use the ‘NbClust’ and ‘kmeans’ R functions to implement k-means clustering on the lodgepole pine data. We first load the R libraries we will be using. The ‘NbClust’ function is available in the ‘NbClust’ R library and the ‘kmeans’ function is available in base R. The ‘tidymodels’ and ‘dplyr’ R libraries are also loaded, since we will be using the ‘initial_split’ function (from ‘tidymodels’) to partition the data into training and test sets, as well as the ‘select’ function (from ‘dplyr’) to select specific columns.

```
library(tidymodels)
library(dplyr)
library(NbClust)
```

K-Means Clustering

Let’s see if the observations can be clustered based on their horizontal distance to important landmarks (horizontal_distance_to_hydrology, horizontal_distance_to_roadways, and horizontal_distance_to_fire_points). In practice, we can cluster based on any features we think are relevant; this is where the subjective part of unsupervised learning comes into play, since our analysis is often guided by subject matter knowledge and expertise.

Before we implement k-means clustering, we center and scale the features in the training and test sets, which is critical for k-means clustering for the reasons discussed during the live session. We also only keep the features that will be included in the clustering analysis: horizontal_distance_to_hydrology, horizontal_distance_to_roadways, and horizontal_distance_to_fire_points.

```
data <- read.csv("lodgepole_data.csv")

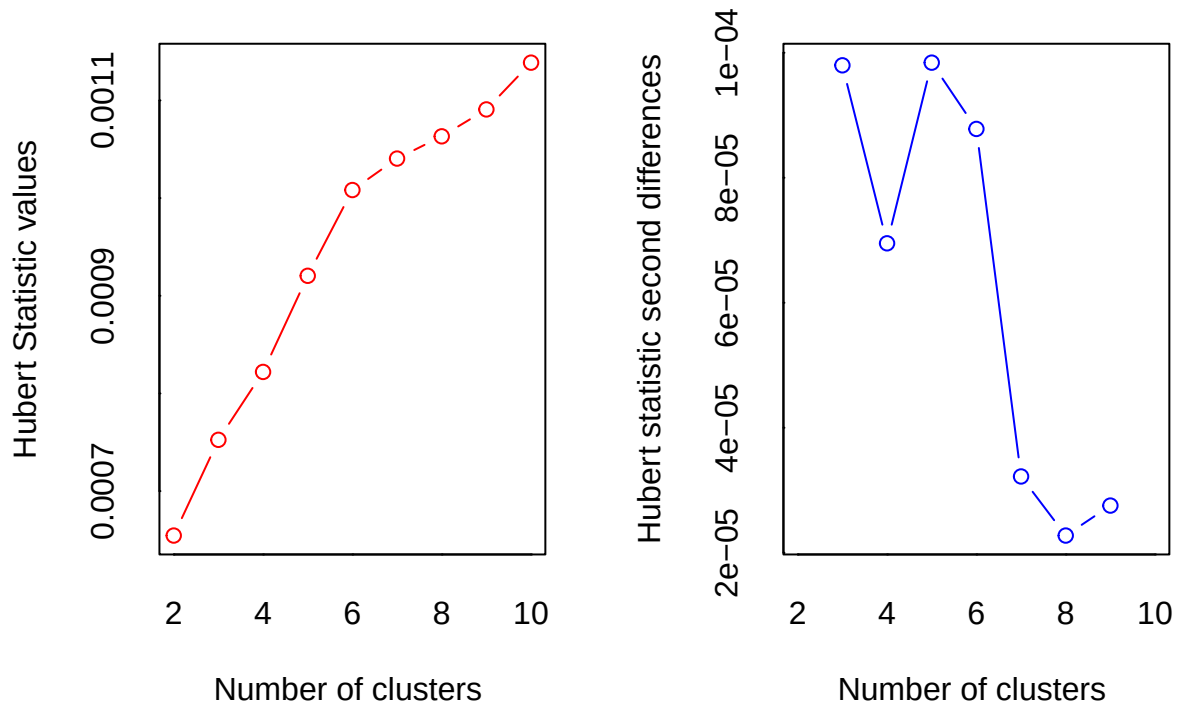
split <- initial_split(data, prop = 0.75, strata = lodgepole_pine)
training_set <- training(split) %>% select(horizontal_distance_to_hydrology,
                                          horizontal_distance_to_roadways,
                                          horizontal_distance_to_fire_points)
test_set <- testing(split) %>% select(horizontal_distance_to_hydrology,
                                     horizontal_distance_to_roadways,
                                     horizontal_distance_to_fire_points)

# center and scale the features
test_set <- scale(test_set,
                  center = apply(training_set, 2, mean),
                  scale = apply(training_set, 2, sd))
training_set <- scale(training_set)
```

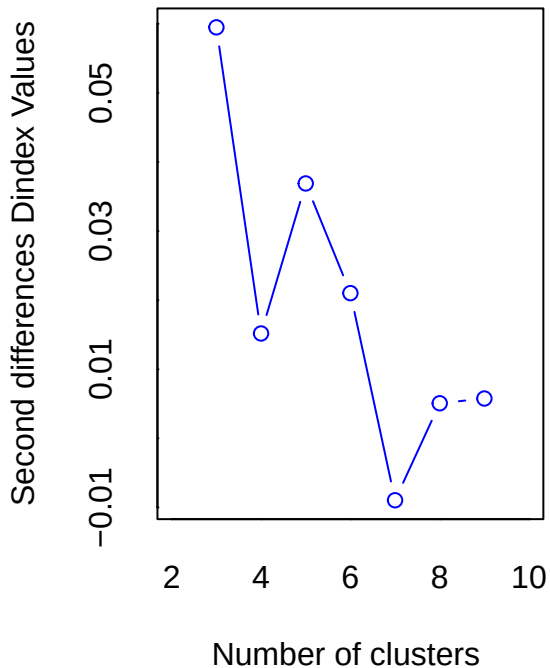
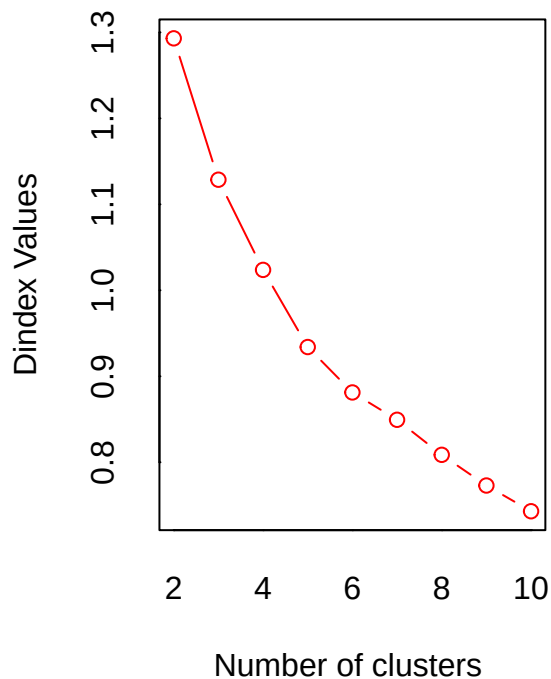
Now that the features have been centered and scaled, we implement k-means in R using the ‘NbClust’ and ‘kmeans’ functions. We first use the ‘NbClust’ function to determine if there is an optimal number of clusters according to the 26 criteria incorporated into this function. We use a random sample of 1000 rows of the original training set, since the ‘NbClust’ function can take a long time to run on large datasets. We set the seed for random number generation, so the exact same results will be obtained when run on your machine (otherwise, the results may be different, since the sample of 1000 rows may be different).

```
set.seed(123)
nc <- NbClust(training_set[sample(nrow(training_set), 1000), ],
```

```
min.nc = 2, max.nc = 10, method = "kmeans")
```



```
## *** : The Hubert index is a graphical method of determining the number of clusters.
##       In the plot of Hubert index, we seek a significant knee that corresponds to a
##       significant increase of the value of the measure i.e the significant peak in Hubert
##       index second differences plot.
##
```



```
## *** : The D index is a graphical method of determining the number of clusters.
##           In the plot of D index, we seek a significant knee (the significant peak in Dindex
##           second differences plot) that corresponds to a significant increase of the value of
##           the measure.
##
## *****
## * Among all indices:
## * 5 proposed 2 as the best number of clusters
## * 5 proposed 3 as the best number of clusters
## * 6 proposed 4 as the best number of clusters
## * 2 proposed 5 as the best number of clusters
## * 1 proposed 6 as the best number of clusters
## * 1 proposed 7 as the best number of clusters
## * 1 proposed 9 as the best number of clusters
## * 2 proposed 10 as the best number of clusters
##
##           ***** Conclusion *****
##
## * According to the majority rule, the best number of clusters is 4
##
## *****
```

As can be seen in the output of the 'NbClust' function, the majority of the 26 criteria voted for 4 clusters to use for k-means. Now, to obtain the 4 clusters, we use the 'kmeans' function with the 'centers' argument set to 4.

```
km_clusters <- kmeans(training_set, centers = 4)
```

After calculating the clusters, we can append the cluster labels to the training set. Now, we can use the cluster labels as a categorical feature in a supervised learning model (such as a dense feed-forward neural network). As with any other categorical feature, we need to encode the cluster label (such as using one-hot encoding) before applying a supervised learning model.

```
cluster_number <- data.frame(cluster_number = km_clusters$cluster)
training_set_with_cluster_num <- cbind(training_set, cluster_number)
head(training_set_with_cluster_num)
```

```
## horizontal_distance_to_hydrology horizontal_distance_to_roadways
## 1 -0.40463492 -1.046524
## 2 -0.81744351 1.584631
## 3 -1.06892461 1.616371
## 4 -1.12586372 1.508197
## 5 0.04613308 0.798912
## 6 -0.12942919 -1.357443
## horizontal_distance_to_fire_points cluster_number
## 1 2.69366543 4
## 2 -0.15819160 2
## 3 -0.06780236 2
## 4 0.23553779 2
## 5 0.93950143 4
## 6 0.34201325 1
```

In this lab, we used the ‘NbClust’ and ‘kmeans’ functions to implement k-means clustering. However, several different methods for implementing clustering exist and are also implemented in R. Look into the different methods, as well as other arguments, that are implemented in the ‘NbClust’ function discussed above.

Exercises

- 1) Why is it good practice to center and scale before applying k-means clustering?
- 2) Print the cluster sizes and centers to the R console. Include a screenshot of the output.
- 3) Use the `aggregate()` function to calculate the mean of each variable within each cluster. Include a screenshot of the output.
- 4) Assign the observations in `test_set` to the clusters found using the sample of the observations in `training_set` above. Include a screenshot of the output. Hint: a function in the ‘clue’ R library can be used for this.